Golden Moments FC — Sistema de tiro y pase

Oct 20, 2026 · @andoni

Resumen

Este documento explica el sistema de tiro y pase contextual de los highlights de Golden Moments FC. El jugador apunta con el ratón en la vista isométrica y el juego dibuja en tiempo real la trayectoria del balón, avisa si un rival puede cortarla y, al soltar, golpea con física realista.

El código está escrito en **Unity C#** (15 scripts, unas 1.700 líneas) y se entrega en el paquete `GoldenMoments_Unity.zip` (un solo paquete, ya integrado) junto a un README de montaje.

Qué resuelve

1. **Trayectoria en pantalla:** rasa o bombeada según la acción, calculada a partir de la posición del cursor respecto al jugador con balón.
2. **Fuerza:** según la distancia al destino (pase raso) o el tiempo que se mantiene pulsada la tecla (pase al hueco, centro y tiro).
3. **Física del balón:** gravedad, rozamiento del aire y del césped, botes, y choque con postes, larguero y red. Sin efecto por ahora.
4. **Rivales en la línea de pase:** aviso previo en la trayectoria y choque real durante el vuelo.

Decisiones clave

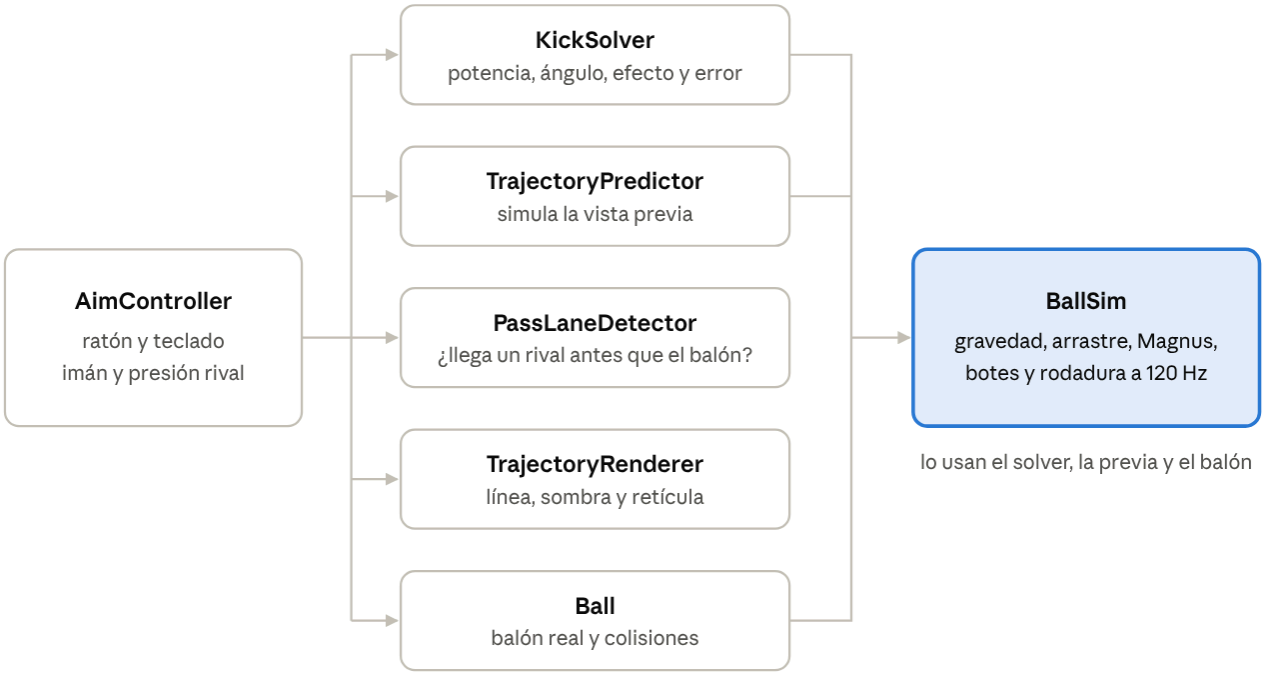
- **Un único integrador físico.** La misma función mueve el balón real, dibuja la vista previa y calcula la potencia necesaria. Lo que el jugador ve es lo que ocurre; solo el error del futbolista separa la previsión del resultado.
- **Sin el Rigidbody de Unity.** El balón se integra a mano en pasos fijos de 1/120 s, así el resultado es idéntico a cualquier framerate y reproducible con la semilla del partido.
- **Valores alineados con los documentos de diseño.** Ventanas de carga, zona óptima, imán de compañero y fórmula de error salen de la especificación técnica de controles y del sistema de Jugadas Destacadas.

Cambio del 27 oct 2026: sin efecto. Se ha quitado el efecto (Magnus, rueda del ratón, liftado y cortado) para dejar el tiro estable y predecible mientras se ajustan porterías, portero y defensa. El balón solo tiene gravedad, arrastre, botes y rodadura. Si vuelve, entrará como un término $k_m(\omega \times v)$ en `BallPhysics.Integrate`, sin tocar el resto.

Arquitectura

Cada fotograma, el controlador de apuntado lee la entrada, pide al solver la velocidad y la dirección del golpeo, simula la trayectoria, evalúa la línea de pase y la dibuja. Al soltar, entrega el golpeo al balón, que usa el mismo integrador.

Un solo integrador mueve el balón, dibuja la previa y calcula el golpeo



módulos del sistema · 7 piezas, 1 integrador

Como el solver, la vista previa y el balón comparten BallSim, la trayectoria dibujada y la real no pueden divergir.

Archivo	Qué hace
`AimController.cs`	Lee ratón y teclado, aplica el imán de compañero, calcula la presión rival, coordina vista previa y ejecución
`KickSolver.cs`	Curva de carga, penalización fuera de zona óptima, error σ y búsqueda de la potencia o el ángulo
`BallSim.cs`	Integrador físico: gravedad, arrastre, botes, rodadura, postes, larguero y red
`TrajectoryPredictor.cs`	Simula el balón hacia delante para la vista previa

Archivo	Qué hace
`PassLaneDetector.cs`	Compara cuándo llega cada rival y el receptor a cada punto de la trayectoria
`TrajectoryRenderer.cs`	Línea coloreada por riesgo, sombra en el suelo, retícula de dispersión, anillo del receptor
`Ball.cs`	Balón en escena: paso fijo, posesión y colisiones con jugadores y postes
`PlayerAgent.cs`	Atributos, atributo efectivo, velocidad, reacción y alcance a pie, de cabeza y de portero
`GoalMouth.cs`	Apuntado sobre el plano de la portería para tirar con altura
`BallPhysicsSettings.cs`, `KickConfig.cs`	Parámetros ajustables como ScriptableObjects
`KickTypes.cs`, `BallState.cs`	Tipos de datos compartidos
`ChargeBarUI.cs`, `TopDownMover.cs`	Barra de carga y movimiento de prueba

Apuntado y trayectoria en pantalla

El cursor no marca una dirección relativa al jugador, sino un punto real del campo. El juego proyecta la posición del ratón desde la cámara isométrica hasta el césped y usa ese punto como destino.

Qué se hace con el cursor

- **Pases y centros:** el destino es el punto del césped bajo el cursor.
- **Tiros:** si el cursor está sobre la portería, la proyección se hace sobre el plano de la línea de gol, así el jugador elige también la altura (escuadra, a ras de suelo). Fuera de la portería, se apunta a 0,6 m de altura.
- **Imán de compañero:** si el cursor queda a 1,5 m o menos de un compañero, el destino se fija en él. En el pase raso va adelantado según la carrera del receptor, como hace el código; en el pase al hueco, 2 m por delante de él.

Rasa o bombeada

Entrada	Trayectoria
Clic izquierdo	Rasa, pase automático al destino

Entrada	Trayectoria
Clic derecho mantenido	Rasa, la carga fija la distancia
Q mantenido	Bombeada a 20°, centro
Espacio mantenido	Tiro: el ángulo se calcula para pasar a la altura apuntada
Ctrl + cualquiera	Globo o vaselina, bombeada a 38–48°

Qué ve el jugador

- **Línea de trayectoria:** la simulación completa del vuelo y la rodadura, recalculada cada fotograma con la carga actual.
- **Sombra en el suelo:** en balones aéreos, una línea oscura sobre el césped deja leer la altura en vista isométrica.
- **Retícula de dispersión:** un círculo en el destino que cubre el 95 % de los golpeos posibles (±2σ). Crece con la fatiga, la presión rival y una mala carga.
- **Anillo bajo el receptor** cuando el imán está activo.
- **Color de la línea:** verde #3DDC84 si es segura, naranja #FF8A4C si un rival llega justo, roja #FF4D5E desde el punto en que la cortan.

Cálculo de la fuerza

La fuerza sale de dos fuentes según la acción: la **distancia** al destino o el **tiempo de carga** de la tecla.

Por distancia

En el pase raso, el centro y los globos, el juego no pregunta cuánta fuerza: la calcula. Busca por bisección (20 iteraciones) la velocidad inicial que, simulada con la física real, deja el balón justo en el destino:

- **Pase raso:** el balón debe llegar a 10 m/s, así el receptor lo recibe con fuerza.
- **Centro y globo:** el primer bote debe caer en el punto apuntado.
- **Tiro:** la potencia sale de la carga, y lo que se calcula es el **ángulo** para que el balón pase a la altura apuntada.

Cada acción tiene un alcance máximo según el atributo del jugador. Un pase raso llega a 28 m con pase 1 y a 45 m con pase 100. Si el cursor está más lejos, el pase se queda corto en esa dirección.

Por tiempo de carga

Mientras se mantiene la tecla, la carga crece así:

```
c(t) = \mathrm{clamp}\left(\frac{t - t_{min}}{t_{max} - t_{min}},\ 0,\ 1\right)^{0.8}
```

El exponente 0,8 da más control en la zona media de la barra. El tiempo es real, así que la carga no se ralentiza durante el tiempo bala.

Acción	Ventana de carga	Qué controla la carga
Pase al hueco	0,12–0,90 s	Distancia: de 6 m al alcance máximo
Centro	0,15–1,00 s	Precisión: la zona óptima se coloca según la distancia al cursor
Tiro	0,15–1,10 s	Velocidad: de 12 m/s a 24–34 m/s según potencia

Zona óptima y sobrecarga. En el tiro, soltar entre el 65 % y el 92 % de la barra da la máxima precisión. Fuera de ese tramo el error crece hasta ×2,5, y por encima del 92 % el balón además se eleva hasta 2,5 m extra.

Error del jugador

Al soltar, la dirección se desvía con un error gaussiano (en altura también en tiros y balones aéreos):

```
\sigma = \sigma_{base} \cdot (1.6 - 0.01\,H_{ef}) \cdot (1 + 0.6\,P) \cdot g(c)
```

H_ef es el atributo efectivo de la acción (pase, centro o remate), que ya incluye fatiga, moral y química; P la presión de rivales a 2 m o menos y g(c) la penalización por carga. El error usa una semilla por partido, así que es reproducible.

Física del balón

El balón se integra en pasos fijos de 1/120 s con Euler semi-implícito, que es estable y barato. Tiene dos estados: en el aire y rodando.

En el aire

```
a = g \underbrace{- k_d |v| v}_{\text{arrastre}}
```

- **Gravedad:** 9,81 m/s² hacia abajo.

- **Arrastre del aire:** cuadrático con la velocidad (k_d ≈ 0,0133, calculado con la densidad del aire, el coeficiente de arrastre y la sección del balón). Frena más los tiros fuertes.

Botes

Al tocar el suelo, el balón conserva el 62 % de su velocidad vertical y el 82 % de la horizontal. Cuando la velocidad vertical del rebote baja de 0,9 m/s, pasa a rodar.

Rodando

- Deceleración constante de 1,6 m/s² por el césped más el arrastre del aire.
- Se para por debajo de 0,05 m/s.

Valores comprobados

Como aquí no hay Unity, el integrador se replicó en una simulación numérica para validar las constantes:

Caso	Resultado
Pase raso de 10 / 20 / 35 m	Sale a 12,9 / 16,0 / 21,0 m/s y llega en 0,9 / 1,6 / 2,4 s
Centro de 25 / 40 m a 20°	Sale a 21,4 / 29,0 m/s, vuela 1,5 / 2,0 s y sube a 2,7 / 4,5 m
Tiro a 26,9 m/s (carga 0,78) desde 11 / 16 / 25 m, apuntado a 1,20 / 2,1 / 0,3 m de altura	Cruza la línea a la altura pedida y es gol en los tres casos; la red lo retiene (ver Porterías)

La primera versión dejaba llegar el pase raso a 3,5 m/s y tardaba 3 s en recorrer 20 m, demasiado lento para un arcade. Se subió la velocidad de llegada a 10 m/s.

Porterías: postes, larguero y red

La portería es geometría del núcleo (`GoalFrame.cs`), no un collider de Unity, para que el resultado sea igual en 2D, en 3D y en la repetición.

Pieza	Medida	Modelo
Boca	7,32 × 2,44 m entre caras interiores	Gol cuando el balón entero cruza la línea entre los postes y bajo el larguero

Pieza	Medida	Modelo
Postes y larguero	Cilindros de 12 cm de diámetro (radio 0,06 m)	Segmentos con radio; choque balón–cilindro
Red	2 m de fondo a ras de suelo, 1,5 m de alto atrás	Cinco paneles: fondo, dos laterales y techo inclinado

Rebote en el palo

En el punto de contacto se separa la velocidad en la componente normal al palo y la tangencial. La normal se invierte y conserva el 70 % (`PostRestitution` = 0,70); la tangencial conserva el 90 % (`PostTangentKeep`). Por eso un tiro que roza el palo por dentro sigue hacia la red y uno que lo golpea de lleno vuelve al campo. El choque se comprueba en 4 subpasos por paso de 1/120 s: a 35 m/s el balón avanza 7 cm por subpaso, menos que el diámetro del palo, así que no lo atraviesa.

La red retiene el balón

Dentro de la portería el balón choca con los paneles con restitución 0,06 (`NetRestitution`), conserva solo el 30 % de la velocidad al rozarlos (`NetTangentKeep`) y sufre 3 m/s² de frenado extra (`NetInsideDrag`). El resultado es que el balón muere dentro y no sale rebotado. En la vista 2D la red se abomba con un muelle amortiguado (`NetMesh2D`), solo visual.

Caso validado	Resultado
Tiro a 26,9 m/s a la red	Llega a x = 54,39 m (fondo a 54,5), no vuelve a cruzar la línea y se para dentro
Poste a 25 m/s	Impacto normal de 19,5 m/s; el balón vuelve al campo
Poste a 35 m/s (prueba de atravesar)	Detectado, impacto de 27,7 m/s
Larguero a 28 m/s	Detectado
Roce interior del poste	Poste y después gol

Probabilidad de dar en el palo

No hay una tirada de «palo sí o no»: el palo sale de la física, a partir del punto apuntado y del error σ del tiro. `ShotOutcomeOdds.Estimate` integra la normal del error (lateral σ y vertical 0,6·σ) sobre las franjas de los palos y da las probabilidades de dentro, poste,

larguero y fuera. Sirve para la IA (elegir dónde apuntar) y para ajustar.

Tirador y destino	Poste	Larguero	Dentro
Remate 50, 16 m, junto al poste	10,3 %	—	70 %
Remate 85, 16 m, junto al poste	11,4 %	—	78 %
Remate 50, 16 m, escuadra	9,4 %	10,5 %	—
Remate 50, 16 m, al centro	—	2,5 %	—
Remate 85, 16 m, al centro	—	0,3 %	—

Apuntar junto al palo da alrededor de un 10 % de palos a cualquier nivel: el mejor rematador mete más dentro, pero sigue arriesgando el palo porque apunta más cerca.

Rivales en la línea de pase

La detección trabaja en dos momentos: antes de golpear, como aviso en la trayectoria, y durante el vuelo, como choque físico real.

Aviso antes de golpear

Para cada punto de la trayectoria prevista se conoce dónde está el balón y en qué instante pasa por ahí. Para cada rival se calcula cuánto tardaría en llegar a ese punto:

```
t_{rival} = t_{reacci\acute{o}n} + \frac{\max(0,\ distancia - alcance)}
{velocidad}
```

- **Reacción:** de 0,40 s a 0,18 s según la anticipación del rival.
- **Velocidad:** v_carrera con su velocidad efectiva (diccionario §5.3), sin factor extra de fatiga.
- **Alcance según la altura del balón:**
  - 0,9 m a pie, si el balón va a 1 m o menos;
  - 0,6 m de cabeza, hasta 2,35 m;
  - 1,8 m con las manos del portero, hasta 2,9 m;
  - por encima de eso, el rival no cuenta: el balón le pasa por arriba.

Si el rival llega antes que el balón, el pase está **interceptado** y la línea se pinta en rojo desde ese punto. Si no, el riesgo es riesgo = σ((t_llegada − t_bal) / 0,15 s) con el rival más rápido (diccionario §5.6): < 0,33 verde #3DDC84, < 0,66 naranja #FF8A4C y el resto rojo #FF4D5E.

El receptor se evalúa con la misma regla: si llega antes que cualquier rival, el pase se considera recibido y la evaluación termina ahí. En los tiros no hay receptor, y el portero cuenta como un rival más.

Choque durante el vuelo

El balón real lanza un barrido esférico (SphereCast) en cada paso de 1/120 s:

- **Contra postes, larguero o vallas:** rebota conservando el 70 % de la velocidad normal.
- **Contra un jugador:** si puede controlarlo, se queda con la posesión; si no, el balón se desvía en su cuerpo y pierde velocidad.
  - Un jugador controla balones de hasta 11–24 m/s según su control (atributo que sustituye a firstTouch; en PlayerAgent, pace pasa a velocidad y firstTouch a control) y a 1,2 m o menos de altura.
  - El portero tiene 10 m/s más de margen y alcanza hasta 2,9 m.
- El lanzador no puede tocar su propio balón durante 0,25 s, para que no choque consigo mismo al golpear.

Montaje, ajuste y limitaciones

Requisitos

- Unity 6 LTS.
- Paquete Input System, con la entrada configurada en «Input System Package».
- UI Toolkit para la barra de carga.

Montaje en escena

1. Crear los assets **Ball Physics Settings** y **Kick Config** desde *Create › Golden Moments*.
2. Crear las capas `Player` y `Obstacle`, y poner postes, larguero y vallas en `Obstacle`.
3. **Balón:** esfera de radio 0,11 con el componente `Ball`, sin Rigidbody.
4. **Jugadores:** cápsula en la capa `Player` con `PlayerAgent`, su equipo y sus atributos.
5. **Porterías:** un `GoalMouth` en el centro de cada línea de gol, con su eje Z hacia el campo.
6. Un objeto con `AimController` y `TrajectoryRenderer`, que necesita un `LineRenderer`. La sombra, la retícula y el anillo son opcionales.
7. Para empezar en posesión, llamar a `ball.SetOwner(jugador)` al lanzar el highlight.

Ajuste rápido

Quiero…	Parámetro
Pases más rápidos	`arriveSpeed` (KickConfig) o `rollingDecel` (física)
Más o menos rebote en postes y red	`PostRestitution` y `NetRestitution` (asset *Campo, balón y porterías*)
Tiros menos precisos	`sigmaShot`, `pressureK` y `maxChargePenalty`
Que los rivales corten más pases	`footReach` o la anticipación de los rivales
Vista previa más ligera	`previewStride` o `solverIterations`

Limitaciones

- **Sin compilar en Unity.** El código está revisado y la física validada numéricamente, pero no se ha compilado dentro de un proyecto. Conviene revisar la consola en la primera importación.
- **La vista previa no muestra rebotes** en postes ni en jugadores; esos los resuelve el balón en tiempo real.
- **Rivales estáticos en la previsión.** El detector usa la posición actual de los rivales, sin predecir hacia dónde corren. Para la IA convendrá usar su posición extrapolada.
- **Movimiento de prueba.** `TopDownMover` solo sirve para probar; se sustituirá por el controlador definitivo.
- Se resuelven en el vertical slice: compilar en Unity 6 LTS, extrapolar la posición de los rivales en el detector de líneas de pase y añadir rebotes a la vista previa.